

OIKION

oikion.com

Unified Import Engine

Product & Technical Specification

Version 1.0 • March 2026

Classification: Internal – Confidential

Status: Approved for Development

This document defines the full specification for the Oikion Unified Import Engine. It is intended for the development team and should be treated as the single source of truth for implementation scope, UX flows, and architectural decisions.

1. Executive Summary

The Unified Import Engine is a core infrastructure feature that allows Oikion users to import data from any spreadsheet file (CSV, XLSX, XML) and have the system automatically infer, split, and create records across all three core entities - Properties, Clients, and Mandates - while establishing the correct relationships between them.

1.1 Strategic Context

Competitor analysis of the top 5 Greek real estate CRM platforms reveals a significant gap in the market:

- iHomeCRM: No self-service import; export limited to local file saves
- iList / e-agents workspace (Fortunet Hellas): Migration handled as a managed white-glove service
- Real Status (iarts): Data migration performed by their team; no self-service import tool
- Qobrix: Per-module CSV import wizard with auto-matching, but requires separate files per entity type
- Estatebud: Migration as a service; no public self-service import tool

None of these competitors offer a unified multi-entity import from a single file. The Oikion Unified Import Engine is a genuine market differentiator that transforms onboarding from a days-long manual process into a minutes-long automated one.

1.2 Core Value Proposition

- Agencies can import their entire business data from a single messy spreadsheet
- The engine auto-detects entity types, maps columns intelligently, and creates linked records
- Supports Greek, English, and Greeklish column headers natively
- Full import history with rollback capability eliminates fear of data corruption
- Template system ensures repeat imports are frictionless
- Batch image import auto-links photos to properties

2. Scope & Entities

2.1 Supported Entities

The import engine operates across three core entities with n:n linking capability:

Entity	Role in Import	Relationship Model
Property	Supply side: physical real estate assets with location, size, price, features	Links to Clients (owner, renter, buyer), other Properties, and Mandates
Client	People/organisations: owners, buyers, renters, contacts	Links to Properties, other Clients, and Mandates
Mandate	Demand side: buyer/renter briefs with budget, size, location, transaction preferences	Links to Clients (who holds the mandate) and Properties (matched)

All three entities support n:n relationships with each other and with themselves (Client↔Client, Property↔Property, etc.) via the existing Link function.

2.2 Supported File Formats

Format	Extensions	Notes
Excel	.xlsx, .xls	Full multi-tab support with cross-tab relationship detection
CSV	.csv	Single-sheet; UTF-8 encoding with Greek character support
XML	.xml	Structured feeds; portal export compatibility

2.3 Supported Image Formats

For batch image import (linked to Properties):

Format	Handling
JPG/JPEG	Accepted natively; optimized for web delivery
PNG	Accepted natively; optimized for web delivery
WebP	Accepted natively
HEIC	Auto-converted to JPG on upload (common from iPhone cameras)

3. Import Engine Pipeline

The import engine processes files through a structured multi-step pipeline. Each step produces output that feeds into the next, with user intervention points at critical decision moments.

3.1 Pipeline Overview

Step	Name	Type	Description
1	File Upload & Parsing	Automatic	Accept file, detect format, parse tabs/sheets, extract headers and sample data
2	Tab Detection & Cross-Tab Analysis	Auto + Confirm	For multi-tab workbooks: detect shared ID columns across tabs, propose relationship graph
3	Column Classification	Auto + Adjust	Classify each column into an entity (Property, Client, Mandate) and map to a specific field
4	Multi-Client Column Detection	User Input	If multiple client-type columns detected (e.g. owner + renter), prompt user to assign client types
5	Duplicate Detection & Preview	Progressive + Review	Query existing Oikion data, flag potential duplicates, show dry-run summary
6	Conflict Resolution	User Input	Present flagged rows (within-file and against-database duplicates) for user decision
7	Commit Import	Automatic	Create entities, establish links, generate friendly names, log import job
8	Post-Import Summary	Informational	Show results: created, linked, skipped, failed; option to export failed rows

3.2 Step 1: File Upload & Parsing

The engine accepts files via the existing Import button on Properties, Clients, or Mandates pages. Regardless of which page initiated the import, the unified engine processes all entity types found in the file.

Parsing Rules

- Detect file encoding (UTF-8, Windows-1253 for Greek, ISO-8859-7) and normalize to UTF-8
- For XLSX: enumerate all tabs/sheets; extract headers (first row) and sample data (first 10 rows) per tab
- For CSV: detect delimiter (comma, semicolon, tab) automatically
- For XML: parse structure and flatten to tabular representation
- Strip empty rows and columns; detect and skip summary/total rows at the bottom
- Identify header row (first non-empty row by default; allow user override if headers are not in row 1)

3.3 Step 2: Tab Detection & Cross-Tab Analysis

For multi-tab XLSX workbooks, the engine analyzes relationships between tabs:

Auto-Detection Logic

1. Scan each tab's headers for ID-like columns (e.g. `client_id`, `property_id`, `id`, `κωδικός`)
2. Check if the same ID column name appears in multiple tabs — this suggests a foreign-key relationship
3. Check if values in an ID column of Tab A appear as values in a column of Tab B
4. Propose a relationship graph: e.g. "Tab 'Properties' links to Tab 'Clients' via 'client_id' column"

User Confirmation

The detected relationships are presented to the user for confirmation. The user can:

- Accept the proposed relationships
- Modify the linking columns
- Mark tabs as independent (no cross-tab linking)
- Skip specific tabs entirely

3.4 Step 3: Column Classification & Mapping

This is the core intelligence layer. The engine classifies each column header into an entity type and maps it to a specific Oikion field.

3.4.1 Classification Approach

The engine uses a rule-based synonym dictionary supporting three languages. No AI/ML is used for column matching — this keeps the system deterministic and debuggable.

Synonym Dictionary Structure

Each Oikion field maintains a list of known synonyms in Greek, English, and Greeklish:

Oikion Field	Entity	Sample Synonyms (EN / GR / Greeklish)
property.address	Property	address, location, διεύθυνση, τοποθεσία, diefthinsi, toposhesia
property.price	Property	price, asking_price, τιμή, τιμή_πώλησης, timi, timi_polisis
property.bedrooms	Property	bedrooms, beds, rooms, δωμάτια, υπνοδωμάτια, dwmatia, ypnodomatio
property.area_sqm	Property	area, sqm, size, square_meters, εμβαδόν, τετραγωνικά, emvadon, tetragwnika
client.name	Client	name, full_name, owner, client, όνομα, επωνυμία, ιδιοκτήτης, onoma, idiokitis
client.phone	Client	phone, mobile, tel, τηλέφωνο, κινητό, telefono, kinito
client.email	Client	email, e-mail, ηλ.ταχυδρομείο, email
mandate.budget_min	Mandate	budget_min, min_budget, ελάχιστο_budget, elachisto_budget
mandate.budget_max	Mandate	budget_max, max_budget, μέγιστο_budget, megisto_budget
mandate.transaction_type	Mandate	transaction, type, είδος_συναλλαγής, eidos_synallagis, buy, rent, sale

The synonym dictionary is extensible. As agencies import files with novel headers, unmatched columns are logged and can be reviewed periodically to expand the dictionary. Templates also serve as a learning mechanism.

3.4.2 Confidence Scoring

Each column mapping receives a confidence score:

Score	Meaning	UX Treatment
90–100%	Exact or near-exact dictionary match	Auto-mapped, shown as green in mapping UI

Score	Meaning	UX Treatment
60–89%	Partial match (substring, fuzzy)	Suggested mapping, shown as yellow; user should verify
0–59%	No confident match	Shown as red; user must map manually or skip

The existing confidence-based mapping UI is retained. The user sees all columns with their proposed mappings and can adjust any mapping before proceeding.

3.4.3 Entity Disambiguation Rules

When a column could belong to multiple entities (e.g. “rooms” could be a Property field or a Mandate preference), the following priority rules apply:

5. If the file contains clear property identifiers (address, coordinates, property_id), then physical attributes (rooms, area, floor) map to Property
6. If the file contains ONLY demand-side fields (budget range, timeline, transaction_type) with no property identifiers, then those fields create a Mandate
7. If contact information is present alongside either, a Client is always created
8. If both property and mandate fields exist in the same row, Property takes priority for shared fields (rooms, area, location) and Mandate gets budget/preference fields

Critical rule: A row with ONLY mandate-like fields (budget, rooms, location preferences) and NO property identifier (no address, no coordinates) creates a Mandate + Client (if contact info present). It does NOT create a Property.

3.5 Step 4: Multi-Client Column Detection

This step activates when the engine detects multiple columns that map to Client-type fields with different semantic roles — for example, a column “prop_owner” and a column “renter” in the same file.

Detection Trigger

The step triggers when two or more columns in the same row map to client.name (or client contact fields) with different source column names suggesting different roles.

User Interaction

During the column mapping phase (before import), the user is presented with an additional panel:

- Each client-type column is listed with a dropdown to assign a client relationship type
- Available types: Owner (Ιδιοκτήτης), Buyer (Αγοραστής), Renter (Ενοικιαστής), Seller (Πωλητής), Contact (Επαφή), Other (Άλλο)
- Default assignment: the first client column is assumed to be “Owner”
- The engine creates separate Client records for each column, with the only difference being the relationship type on the link to the Property

Example

A file has columns: property_address, price, prop_owner, owner_phone, renter, renter_phone

- Column “prop_owner” → User assigns type “Owner”
- Column “renter” → User assigns type “Renter”
- Result: 1 Property + 2 Clients created, each linked to the Property with their respective type

3.6 Step 5: Duplicate Detection & Preview

Before committing any data, the engine performs a comprehensive duplicate analysis against both the file contents and existing Oikion data. This runs as progressive background processing with live updates.

3.6.1 Within-File Deduplication

The engine identifies records that appear multiple times within the uploaded file:

Clients: If the same client name appears in multiple rows (e.g. an owner with 15 properties), the engine creates ONE client and links all properties to it. If a field value differs slightly between rows (e.g. phone number), the specific row is flagged for user review with a message like:

“We have created the client ‘Γιάννης Παπαδόπουλος’. On Row 17, is it the same client? We notice a difference in the phone numbers.”

Properties: Duplicates are detected by matching ID field (if present), or Name + Coordinates + Address combination (depending on available columns).

3.6.2 Against-Database Deduplication

The engine queries existing Oikion records to find potential matches. This happens during the preview step so users see duplicates before committing.

For files with thousands of rows, this analysis runs in the background with a progressive live-updating UI:

- Progress bar: “Analyzing... 450/2,000 rows checked”
- Results appear incrementally as they’re found
- User can review early results while remaining rows are still being processed
- Potential duplicates are highlighted with a summary of matched fields

3.6.3 Dry-Run Summary

Once analysis completes, the user sees a full preview:

Metric	Example
Properties to create	47 new, 3 potential duplicates
Clients to create	23 new, 5 potential duplicates, 2 field conflicts
Mandates to create	12 new
Links to establish	47 Property↔Client, 12 Mandate↔Client
Rows with errors	8 rows (missing required fields)
Rows skipped	2 rows (empty)

3.7 Step 6: Conflict Resolution

All flagged rows and potential duplicates are presented to the user for manual resolution. This is a per-record approval process.

Resolution Options per Flagged Record

Scenario	Options Available
Potential duplicate (existing record in Oikion)	Skip (don't import) Create new record anyway Merge with existing (update fields)
Within-file field conflict (e.g. different phone for same client)	Keep first occurrence Keep this row's value Mark as separate client
Missing required fields	Fill in manually Skip row Import partial record

3.8 Step 7: Commit Import

Once the user confirms, the engine commits all records in a single transactional batch:

9. Create Client records first (since Properties and Mandates link to them)
10. Create Property records, linking to Clients with assigned relationship types
11. Create Mandate records, linking to Clients
12. Establish any cross-entity links (Property↔Mandate, Client↔Client, etc.)
13. Generate intelligent, auto-generated friendly names for all entities (see Section 4)
14. Log the import job with full metadata (file name, timestamp, row count, user, entity counts, template used)

Records that failed validation or were discarded by the user in Step 6 are not imported. Failed rows are available for export (see Step 8).

3.9 Step 8: Post-Import Summary

After the import completes, the user sees a summary screen with the following:

- Total records created per entity type, with links to view them
- Total links established
- List of skipped/failed rows with reasons
- **Export Failed Rows** button: downloads a new spreadsheet containing only the rows that failed, with an added column explaining the failure reason
- Option to navigate to the Imports History page

4. Friendly Name Generation

When importing data, not all records will have a human-readable name. The engine generates smart, intelligent, auto-generated names for entities that lack explicit name fields.

4.1 Naming Rules

Entity	Name Source Priority	Auto-Generated Pattern
Property	1. Explicit title/name column 2. Derived from available data	[Type] [Rooms]BR, [Area/Neighbourhood]e.g. "Apartment 3BR, Kolonaki"
Client	1. Explicit name column 2. Derived from linked Property	[Property Name] Owner (or assigned type)e.g. "Kolonaki 3BR Apartment Owner"
Mandate	1. Explicit name/title column 2. Derived from criteria	[Transaction] [Type] [Area]e.g. "Buy Apartment Glyfada"

Name generation should be contextually intelligent. If a Client record is created from a phone number column alongside a Property, the Client name should reference the Property and the relationship type rather than being a generic placeholder like "Client #47".

5. Import Template System

Users can save column mappings as reusable templates for future imports of similarly-structured files.

5.1 Template Contents

A saved template stores:

- Column-to-field mappings (including entity assignment)
- Multi-client type assignments (if applicable)
- Skipped columns
- Cross-tab relationship definitions (for multi-tab workbooks)
- Template name and optional description

5.2 Template Visibility

- Templates are personal (per-user) by default
- Users can share a template with their entire organization via a "Share with team" toggle
- Shared templates appear in a separate "Team Templates" section in the template picker
- Only the template creator (or an admin) can edit or delete shared templates

5.3 Template Matching

When a user uploads a file, the engine checks if any saved template matches the file's column headers. If a match is found (same headers in the same or similar order), the template is auto-suggested. The user can accept the template, modify it, or start from scratch.

6. Batch Image Import

The engine supports bulk upload of property images with automatic linking to Property records. Images are auto-converted to web-optimized format (JPG) regardless of source format.

6.1 Image Matching Methods

The engine supports four matching methods, tried in priority order:

Priorit y	Method	How It Works
1	Separate mapping CSV	User uploads a CSV file with columns: image_filename, property_id (or property name). Each row maps an image to a property.
2	Folder structure	User uploads a ZIP file where each folder is named after a property ID or property name. All images within a folder are linked to that property.
3	Filename convention	Image filenames contain the property ID or a recognizable property identifier, e.g. PROP_001_01.jpg, PROP_001_02.jpg
4	Manual fallback	If no automatic match is found, unmatched images are shown in a drag-and-drop UI where the user can assign them to properties manually.

6.2 Image Ordering

When multiple images are linked to the same property, the order is determined by:

- Numeric suffix in filename (e.g. _01, _02, _03)
- Alphabetical filename order as fallback
- User can reorder in the preview step via drag-and-drop

6.3 Image Limits & Processing

- Per-property image limit follows the existing Property entity limit in the application
- HEIC files are auto-converted to optimized JPG
- Images are compressed/optimized for web delivery during upload
- Excessively large images (>20MB) are flagged for user attention

7. Import History & Rollback

A dedicated Imports page provides full visibility into past imports and the ability to rollback or delete imported data.

7.1 Imports Page

The Imports page displays a chronological list of all import jobs with the following information per job:

Field	Description
Import ID	Unique identifier for the import job
Date & Time	When the import was executed
Imported By	User who performed the import
Source File	Original filename
Template Used	Name of template (if any)
Records Created	Breakdown: X Properties, Y Clients, Z Mandates
Records Modified	Count of records edited since import
Status	Active Partially Rolled Back Fully Rolled Back

7.2 Rollback Mechanism

Users can roll back an entire import job. The rollback system has two modes:

Simple Mode (Default View)

The user is presented with a summary and two options:

- “Rollback all” — reverts everything including records modified post-import
- “Keep modified, rollback rest” — preserves records the user has manually edited since the import and only deletes untouched records

If any records have been modified since the import, a warning is displayed:

“3 of 47 Properties and 1 of 23 Clients have been modified since this import. These changes will be lost if you choose ‘Rollback all’.”

Advanced Mode (Expandable)

For users who want granular control, the simple view expands into a per-record checklist:

- Each record imported in this job is listed with a checkbox
- Modified records are highlighted with an “edited” badge
- User can select/deselect individual records to include in or exclude from the rollback

- A “Select all unmodified” shortcut is available

7.3 Rollback Effects

- Rolled-back records are permanently deleted from the database
- All links involving rolled-back records are also removed
- The import job status changes to “Fully Rolled Back” or “Partially Rolled Back”
- Rollback is itself logged as an action for audit trail purposes
- Rolled-back import jobs remain visible in the Imports page for reference

8. Error Handling & Partial Imports

The engine is designed to maximize successful imports even when files contain problematic data.

8.1 Partial Import Policy

If N out of M rows are valid, the engine imports the N valid rows and quarantines the remaining rows for user review. The user can:

- Review each failed row and fix issues manually in-app
- Discard all failed rows
- Export failed rows to a new spreadsheet (with an added column explaining each failure reason), fix them externally, and re-import

8.2 Error Categories

Category	Example	Handling
Missing required field	Property row with no address or location	Quarantine; user can fill in missing data
Invalid data format	Phone number with letters, price as text	Attempt auto-correction (strip non-numeric); flag if ambiguous
Encoding issues	Garbled Greek characters (ακίνητο appearing as Î±Î±°Î¬)	Attempt re-encoding from common Greek codepages
Ambiguous entity	Column could be Property or Mandate field	Present both options to user in mapping step
File structure error	No header row, merged cells, pivot tables	Display clear error message; suggest file cleanup steps

9. Architecture Notes

High-level architectural guidance for the development team.

9.1 Processing Model

Given the requirement to handle thousands of rows with progressive duplicate detection, the import engine **MUST** process server-side. The pipeline:

- File upload: client-side parsing for preview (first 10 rows) using a lightweight library, then full file sent to server
- Steps 2–6: server-side processing with WebSocket or SSE for progressive updates to the client

- Step 7 (commit): server-side transactional batch write
- The engine should be idempotent — if a commit fails partway through, it should be safe to retry

9.2 Abstraction & Extensibility

The import engine should be designed with extensibility in mind:

- **Source abstraction:** The file parser should be behind an interface (e.g. ImportSource) so future sources (API push, copy-paste, OCR) can be added without changing the pipeline
- **Entity abstraction:** Adding a fourth entity type in the future should require only adding a new classifier rule set and field mappings, not restructuring the pipeline
- **Synonym dictionary:** Should be stored as configuration (JSON or database), not hardcoded, so it can be expanded without code changes
- **Template storage:** Templates should be persisted per-user and per-organization in the database, not in local storage

9.3 Performance Targets

Metric	Target
File parsing (Step 1)	< 5 seconds for 5,000-row XLSX
Column classification (Step 3)	< 2 seconds for 50 columns
Duplicate detection (Step 5)	Progressive; first results within 3 seconds, full scan < 30 seconds for 5,000 rows
Commit (Step 7)	< 10 seconds for 1,000 entities
Image processing	< 2 seconds per image (HEIC conversion + optimization)

10. Entry Points & UX Integration

10.1 Entry Points

The Unified Import Engine is accessible from:

- Properties page → Import button
- Clients page → Import button
- Mandates page → Import button
- Imports History page (for re-imports or template-based imports)

Regardless of entry point, the same unified engine is invoked. The entry point may pre-select the “primary” entity for confidence scoring, but the engine always processes all entity types found in the file.

10.2 Navigation

The Imports History page should be accessible from:

- Main navigation / Settings area
- Post-import summary screen (direct link)
- Each entity page (Properties, Clients, Mandates) could optionally show a subtle indicator if records on the page came from an import

11. Decision Log

Key product and technical decisions made during specification:

#	Decision	Rationale
D 1	Rule-based synonym dictionary (no AI for column matching)	Deterministic, debuggable, no API cost; dictionary is extensible over time
D 2	Multi-client columns resolved during mapping phase	Earlier is better — affects entity creation logic downstream
D 3	Mandate-only rows create Mandate + Client (never Property)	Clear separation of supply vs demand; prevents phantom property records
D 4	Progressive duplicate detection with live updates	Thousands of rows require async processing; users shouldn't stare at a spinner
D 5	Rollback: simple mode default + expandable advanced mode	Covers 90% of cases simply; power users get granular control when needed
D 6	Templates personal by default, shareable to org	Prevents clutter; agency admins can curate shared templates

#	Decision	Rationale
D 7	Image matching: 4 methods in priority order with manual fallback	Agencies have no standard image naming; must handle everything
D 8	Default client relationship type: Owner	Most common real-world case; reduces clicks for the majority scenario
D 9	Server-side processing with WebSocket/SSE progress	Client-side can't handle thousands of rows + database queries
D 10	Source abstraction behind ImportSource interface	Future-proofs for API push, copy-paste, OCR without pipeline changes

12. Out of Scope (Future Considerations)

The following are explicitly out of scope for this build but should be architecturally accommodated:

- AI-powered column matching (can be layered onto the synonym dictionary later)
- Copy-paste import from clipboard (ImportSource interface supports this)
- OCR import from scanned documents or photos
- API push from external systems (ImportSource interface supports this)
- Scheduled / recurring imports (e.g. nightly sync from an external source)
- Import from specific competitor platforms (no standard export formats exist currently)
- Bulk update mode (importing to update existing records in place, without creating new ones)

End of Specification — Oikion Unified Import Engine v1.0

For questions: hello@oikion.com